

PART VII — PRODUCTION, DEPLOYMENT, AND BEYOND

Chapter 41

Deployment

“The pipeline that answers a query correctly in a notebook is not yet a system anyone else can rely on.”

Chapter 41 — Deployment

Every prior chapter of this book ran its pipeline as a script or a CLI session. This chapter covers the last step that turns it into something another program — a frontend, a Postman client, a second engineer's test — can actually call: a FastAPI service wrapped around each pipeline, a real feedback-persistence backend that survived a genuine scaling problem, and a mechanism for overriding pipeline behavior per request without a restart. As in earlier chapters, the material here is a mix of what this project actually built and general production practice it never had reason to build — this chapter is explicit throughout about which is which.

The two most fully realized parts of this chapter are also its most concretely sourced: `app/api.py` and `app_workflow/api.py`, running the LangChain and LangGraph pipelines side by side on ports 8000 and 8001, and the MongoDB migration this project's feedback layer went through once the original JSONL ledger (Chapter 29) could no longer keep up. Both come with real bugs — a duplicate-key race, a replica-set requirement, an index-definition mismatch — that are worth more to a reader than a clean textbook description of either mechanism would be.

41.1 Wrapping the agent in a FastAPI service

Both pipelines follow the same lifespan pattern, differing mainly in what gets built at startup. `app/api.py`'s `lifespan` handler constructs an `EmbeddingManager` and `VectorStore`, raising `RuntimeError("Vector store is empty. Run ingest.py first.")` if the index has no documents in it, then builds the retriever, tools, `FeedbackStore`, and `SelfLearner`, storing all of it in a module-level `_ctx` dictionary that request handlers read from. `app_workflow/api.py`'s lifespan is structured slightly differently on purpose: it imports `build_graph` and the `services` module lazily, inside the function body rather than at module load time, specifically — per its own code comment — "so that a `SystemExit` from `services.py` (empty vector store) surfaces as a startup failure rather than a module-level crash," a distinction that matters because a module-level crash can produce a confusing import-time traceback instead of a clean, catchable startup error.

Graceful shutdown in both services is a single deliberately simple endpoint: `POST /quit` sends `os.kill(os.getpid(), signal.SIGTERM)` to the running process and returns `{"status": "shutting down"}`, letting uvicorn's own signal handling perform a clean shutdown rather than requiring an operator to find and kill the process externally.

41.2 Running two pipelines side by side

`app/api.py` serves the original LangChain sequential pipeline on port 8000; `app_workflow/api.py` serves the LangGraph parallel-track pipeline on port 8001. Running both at once, against the same corpus, is what made Chapter 39.5's real cross-pipeline latency comparison possible in the first place — a benchmark run against only one pipeline could describe that pipeline's absolute latency, but only running both side by side against the same queries could produce the roughly 2x speedup figure this book has cited from Chapter 39 onward. Figure 41.1 lays the two services out together.

Two pipelines, two ports, one shared lifespan pattern

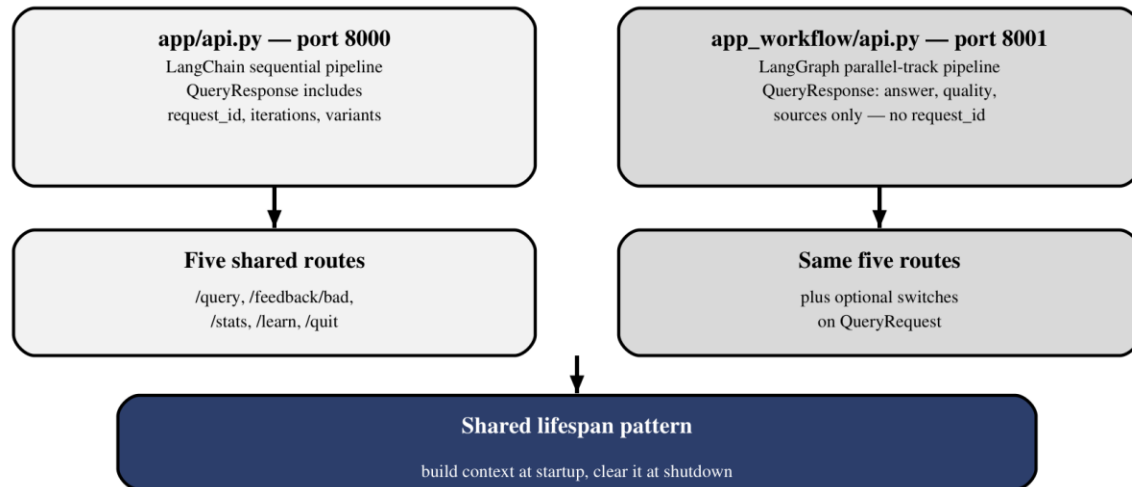


Figure 41.1 — Two independently deployable services share a lifespan pattern and route surface, differing in pipeline implementation and response shape.

41.3 The endpoint surface

Both APIs expose the same five routes: `POST /query`, `POST /feedback/bad`, `GET /stats`, `POST /learn`, and `POST /quit`. `app_workflow/api.py` adds one additional layer of protection its LangChain counterpart doesn't need: a reserved-word guard, `COMMAND_INPUTS = {"bad", "stats", "learn", "exit", "quit"}`, rejecting any of those five literal strings sent as a `/query` payload with an HTTP 400 — a defense against a client accidentally sending a CLI-style command string (Chapter 35's interactive commands) to the wrong endpoint rather than calling the dedicated route for it.

`POST /feedback/bad`, not a bare `/bad`, is the real route name — ADR-047 chose the `request_id`-keyed feedback path specifically because a bare `/bad` route gives no way to identify *which* prior query a thumbs-down applies to once more than one request may be in flight.

41.4 Request/response Pydantic models

The two pipelines' request models diverge specifically around the `switches` mechanism (Section 41.16), but their response shapes diverge more sharply, and the difference is worth internalizing precisely because it is easy to assume the two APIs are interchangeable when they are not. `app/api.py`'s `/query` handler returns whatever `run_agent()` produces plus an injected `request_id`: `answer`, `sources`, `iterations`, `quality`, `document_chunks`, `learned_qa_chunks`, `variants`, and `request_id`. `app_workflow/api.py`'s `/query` handler returns a deliberately narrower shape:

```
return {"answer": answer, "quality": quality, "sources": sources}
```

There is no `request_id` in the LangGraph response at all — a direct, load-bearing consequence of the two pipelines' different feedback mechanisms. `app/api.py`'s `FeedbackRequest` requires a `request_id` because `mark_bad()` needs to know exactly which historical interaction a thumbs-down applies to; `app_workflow/api.py`'s `FeedbackRequest` has no `request_id` field at all, relying instead on an in-memory `mark_last_bad()` sidecar that assumes feedback always refers to the most recent query on that connection. A client written against one pipeline's response contract will silently break if pointed at the other without adjustment — this is a genuine integration hazard, not just a documentation gap.

41.5 Postman setup for parallel-pipeline testing

Common pitfall — Assuming a documented testing workflow still has its artifacts

This project's own development record mentions a hand-written ``API_ENDPOINTS.txt`` — full documentation of every endpoint across both APIs, including request/response JSON shapes and error codes — but neither that file nor any Postman collection or environment file exists anywhere in this repository's current tree. The workflow this section describes is general practice, not a reproduction of a specific project artifact.

The general pattern worth knowing regardless: one Postman collection defining the shared route shapes (``POST /query``, ``POST /feedback/bad``, ``GET /stats``, ``POST /learn``), paired with two environments — one pointed at ``localhost:8000``, one at ``localhost:8001`` — lets the same request bodies be replayed against both pipelines by switching environments rather than duplicating every request. Given Section 41.4's real response-shape divergence, a shared collection's response-assertion scripts need to branch on which environment is active, or accept only the fields both pipelines' responses share.

41.6 Streaming responses to the client

Common pitfall — Assuming this project streams tokens to the client

Neither ``app/api.py`` nor ``app_workflow/api.py`` implements streaming — an exhaustive search for ``StreamingResponse``, ``EventSourceResponse``, or server-sent-events patterns across the entire codebase returns nothing. Both ``/query`` handlers run the full pipeline behind ``await asyncio.to_thread(...)`` and return one complete JSON response only once the entire agent loop, including every retry and every compression pass, has finished.

This is a real, honest limitation worth naming directly: given Chapter 39.13's own measured latency data — individual compression calls running past twelve minutes in the long tail — a client waiting on this project's non-streaming ``/query`` endpoint can genuinely wait minutes with no intermediate feedback. FastAPI's ``StreamingResponse``, paired with a pipeline redesigned to yield intermediate tokens or stage-completion events as they happen rather than only a final answer, is the standard fix for exactly this user-experience gap — a real architectural change this project never made, not a small addition.

41.7 Persistent vector stores in production

This project's vector store has always been a self-hosted, embedded ChromaDB instance backed by a local ``VECTOR_STORE_PATH`` — no managed vector database (Pinecone, Weaviate, or similar) appears anywhere in its codebase or research ledger. ``learned_qa_store.py``'s collection-factory design, covered in Chapter 32, includes a real live-migration case study worth knowing here too: 374 entries were successfully moved from an L2-distance collection to a cosine-distance collection without data loss, demonstrating that a self-hosted embedded store is not necessarily locked into its original distance metric forever, even without a managed provider's migration tooling.

The managed-versus-self-hosted tradeoff itself is genuine production guidance rather than something this project's own history settled: a managed vector database trades operational simplicity (no local disk to manage, built-in replication, a support contract) for a recurring cost and a dependency on a third party's uptime; a self-hosted embedded store like this project's ChromaDB instance trades that simplicity for full control and zero per-query cost, at the price of the team owning backup, replication, and capacity planning themselves.

41.8 Stateless web frontends and session handling

The response-shape divergence Section 41.4 covers has a direct session-handling consequence a frontend integrating with both pipelines has to design around: ``app/api.py``'s ``request_id``-based feedback model is naturally stateless from the frontend's perspective — the frontend simply stores the ``request_id`` it was handed and replays it later, with no server-side session required. ``app_workflow/api.py``'s ``mark_last_bad()`` sidecar is implicitly stateful in a way that does not survive a load-balanced multi-instance deployment cleanly — "the most recent query" is only well-defined if every request from a given client session reliably lands on the same backend instance, which a naive round-robin load balancer does not guarantee. A frontend or infrastructure team adopting the LangGraph pipeline behind more than one instance needs either sticky sessions or a redesign of that feedback mechanism to be explicitly ``request_id``-keyed the way the LangChain pipeline already is.

41.9 Containerization with Docker

Neither ``app/api.py`` nor ``app_workflow/api.py`` has a Dockerfile or docker-compose configuration anywhere in this repository — both run as bare Python processes today. The one real containerization precedent this project does have lives outside the two main APIs entirely: Chapter 41B's Marker microservice, whose ``Dockerfile`` and ``docker-compose.yml`` are described in detail in ADR-073 and this project's architecture ledger, even though — as Chapter 41B is explicit about — the directory itself is not present in this repository's current tracked snapshot. Containerizing the main query APIs the same way Chapter 41B's microservice was containerized is a natural next step this project's own history points toward but has not yet taken.

41.10 Scaling — replicas, load balancing, shared index

This project's ledgers never discuss horizontal scaling, replica counts, or load balancing — there is no scaling configuration or discussion to cite here, and this section is general production guidance rather than a description of anything this project built. The one scaling-relevant constraint worth carrying forward from earlier sections is Section 41.8's session-handling gap: any horizontal scaling plan for the LangGraph pipeline specifically has to resolve the ``mark_last_bad()`` statefulness problem before adding a second instance behind a load balancer, or feedback attribution will silently break for whichever fraction of requests lands on a different instance than the query that preceded it.

41.11 CI/CD for RAG systems

There is no CI/CD configuration anywhere in this repository — no ``.github/workflows/`` directory, no other CI configuration of any kind. What this project does have, covered in Chapter 37.4, is a **manual** discipline that a CI pipeline would formalize: ``test_output_fixes.py``'s 302-case regression suite over the JSON-repair tier, and Chapter 36C's A/B log-comparison methodology, both currently run by hand rather than gated automatically on every change. A RAG-specific CI pipeline built from this project's own tools would plausibly run the JSON-repair regression suite on every commit and require the A/B methodology's fixed query pair to pass before merging any change to a retrieval threshold — formalizing disciplines this project already practices manually into an automated gate it has not yet built.

41.12 From JSONL ledger to MongoDB

ADR-046 is the deciding record, and its options-considered section is worth knowing because SQLite lost the comparison for a reason relevant to this project's own two-pipeline architecture: MongoDB was chosen over both an improved JSONL format and SQLite specifically because the feedback layer needed genuine concurrent-write safety across two independently running API processes — ``app/api.py`` and ``app_workflow/api.py`` writing to the same feedback store from separate processes is exactly the scenario SQLite's file-level locking handles poorly and MongoDB's client-server model handles natively. The ADR's impact statement states the cutover plainly: ``interactions.jsonl``, ``user_thumbsdowns.json``, and ``failed_variants.json`` are no longer written at all; ``MONGODB_URI`` defaults to ``mongodb://localhost:27017`` and ``MONGODB_DB_NAME`` defaults to ``rag_db``, matching literally in both ``app/db.py`` and ``app_workflow/services/db.py``.

41.13 MongoDB replica sets and why multi-document transactions require them

Definition — Replica set / oplog

A MongoDB replica set is a group of ``mongod`` instances maintaining copies of the same data, coordinated through an operations log (the oplog) that records every write in order. Multi-document ACID transactions require this oplog to exist — a standalone, non-replicated ``mongod`` instance has no oplog at all, and therefore cannot support ``session.start_transaction()`` regardless of how the transaction itself is written.

ADR-048 and this project's own research record the concrete reason a transaction was needed here at all: marking a thumbsdown has to update the original interaction record (setting a ``USER_THUMBSDOWN`` flag) and insert a new entry into the thumbsdowns collection (Chapter 31) as a single atomic unit — if the process crashed between the two writes, the feedback store would be left in an inconsistent state no read of it could recover from cleanly. Because this project runs MongoDB locally rather than against a managed cluster with replication already provided, it satisfies the replica-set requirement with a single-node replica set — ``replSetName: "rs0"`` in ``mongod.cfg``, initiated with ``rs.initiate()`` using ``directConnection=True`` — giving the oplog transactions require without needing a second physical machine for local development.

The actual code, ``app_workflow/services/feedback_store.py``, wraps both writes in exactly the transaction the ADR describes:

```
with get_client().start_session() as session:
    with session.start_transaction():
        # update the original interaction (USER_THUMBSDOWN flag)
        # insert the new thumbsdown record
        ...
```

41.14 DuplicateKeyError as an idempotency guard for LangGraph node retries

BUG-055 is this section's grounding case, and the fix it produced is worth understanding as a general pattern well beyond this one bug. A LangGraph node that writes to the feedback store can, under retry conditions covered elsewhere in this book, run more than once for what is logically the same request. A naive check-then-insert pattern — query for an existing record by `request\_id`, insert only if none is found — has a race window: two near-simultaneous executions of the same retried node can both complete the check before either completes the insert, and both then insert, producing a duplicate. Figure 41.2 contrasts the two approaches directly.

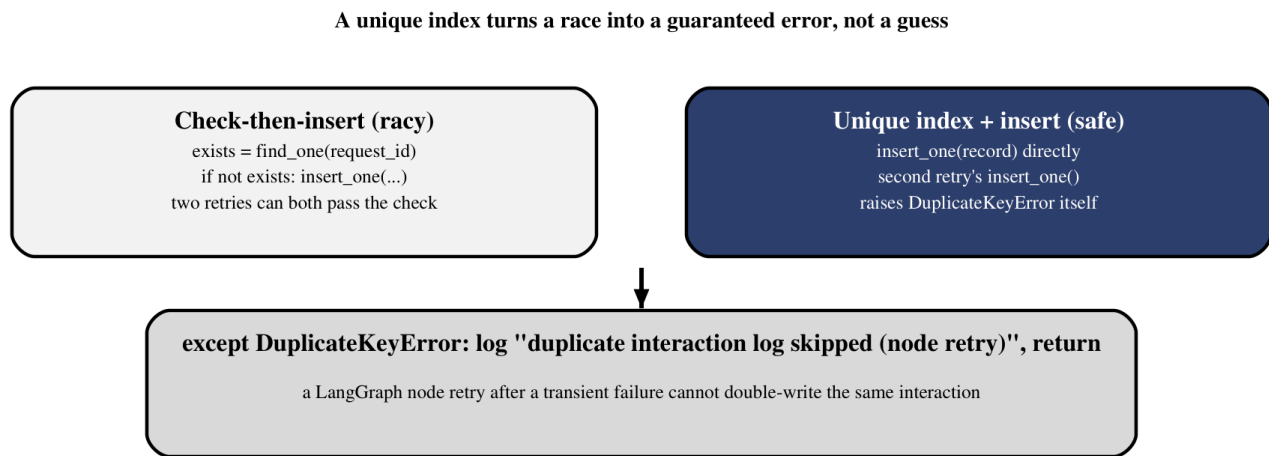


Figure 41.2 — A unique index converts a racy read-then-write into an insert that either succeeds once or fails loudly the second time; there is no window where both branches can win.

The actual fix relies on a unique index on `request\_id` rather than an application-level check at all — `insert\_one()` is called directly, and MongoDB itself is the arbiter of uniqueness:

```
try:
    self._interactions.insert_one(record)
except DuplicateKeyError:
    logger.warning(
        "Duplicate interaction log skipped (node retry): request_id=%s",
        request_id,
    )
    return
```

This is safer than the check-then-insert pattern precisely because there is no window between a check and a write for a second writer to slip through — the database's own unique-index constraint makes the two operations atomic from the perspective of any concurrent caller. A related bug, BUG-054, is worth knowing alongside this one: an `IndexKeySpecsConflict` arose because `app/db.py` and `app\_workflow/services/db.py` created the same `request\_id\_1` index with mismatched `sparse` settings on each side — a reminder that an idempotency guard built on a unique index is only as reliable as every codepath that creates that index agreeing on its exact definition.

41.15 Migration path — copying the flat files into MongoDB

ADR-046's context section names the three flat files this migration replaced and the collections they became: ``interactions.jsonl`` into ``feedback_interactions``, ``user_thumbsdowns.json`` into ``user_thumbsdowns``, and ``failed_variants.json`` into ``failed_variants``. It is worth being precise about what this project actually built versus what a reader might assume: the ADR describes a cutover — the flat files stopped being written the moment MongoDB writes began — rather than a historical backfill tool. No dedicated migration script moving pre-existing flat-file history into the new collections exists anywhere in this codebase. A team performing this same migration on a project with feedback history worth preserving would need to write that backfill script themselves; this project's own transition simply drew a line and began writing to MongoDB going forward.

41.16 Per-request pipeline control

Chapter 36B introduced the twenty ``ENABLE_*`` workflow flags as ``config.py`` defaults, and ADR-071 is what makes them overridable per request without a restart. ``app_workflow/api.py``'s ``QueryRequest`` accepts an optional nested ``switches`` object, built from a ``WorkflowSwitches`` Pydantic model with all twenty flags declared as ``Optional[bool] = None`` — a field left unset simply means "use the ``config.py`` default," never "force off." ``app_workflow/services/switches.py``'s ``resolve_switches()`` performs the actual merge:

```
def resolve_switches(overrides: dict | None) -> dict[str, bool]:
    merged = dict(DEFAULT_SWITCHES)
    if overrides:
        for key, value in overrides.items():
            if key in SWITCH_NAMES and isinstance(value, bool):
                merged[key] = value
    return merged
```

The API handler calls ``switches.model_dump(exclude_none=True)`` before passing overrides through, so an unset field never overwrites a default with ``None`` by accident — only fields the caller explicitly set participate in the merge at all. The resolved dictionary is stored directly on ``GraphState`` under the ``switches`` key at graph construction time, and ``get_switches(state)`` reads ``state.get("switches")`` or `DEFAULT_SWITCHES` throughout the rest of the run, meaning every node in that specific request's graph execution sees the same resolved configuration without needing to re-read ``config.py`` or re-merge overrides itself. This is what lets one team run an A/B comparison — Chapter 36C's methodology, but live in production rather than against saved logs — by sending two requests with different ``switches`` payloads to the exact same running service, with no deployment or restart between them.

It is worth closing this chapter on the same honest note ADR-071's own operational history includes: during the benchmark run that exercised this mechanism most heavily, the local MongoDB replica set became unavailable partway through, and the request-scoped ``switches`` override did nothing to isolate that failure — a per-request configuration knob changes what a request does, not whether the infrastructure it depends on stays up. Part VII closes with exactly this reminder in mind: everything this part of the book has covered — evaluation, observability, performance, deployment — makes a system legible and controllable, but none of it substitutes for the infrastructure underneath actually staying available.